

Práctica 9: Deep Learning

Tratamiento de Señales Visuales, GCID

Domínguez Hernández, Iván

Miranda López, Lucas

Curso 2025–2026

Introducción y objetivo de la práctica

El objetivo de esta práctica es aplicar redes neuronales convolucionales (CNN) para clasificar imágenes de ultrasonido mamario del dataset BUSI (*Breast Ultrasound Images*) [1], siguiendo el flujo del tutorial de MATLAB *View Network Behavior Using t-SNE* [5, 8]. El dataset contiene 780 imágenes distribuidas en tres clases: **benign** (487 imágenes), **malignant** (210) y **normal** (133), junto con sus correspondientes máscaras de segmentación.

El trabajo se divide en dos ejercicios: el primero adapta el tutorial para clasificar imágenes con una red preentrenada mediante *transfer learning* [6] comparándola con clasificadores triviales, y el segundo analiza el impacto de distintos hiperparámetros. La Figura 1 resume el flujo completo del sistema.

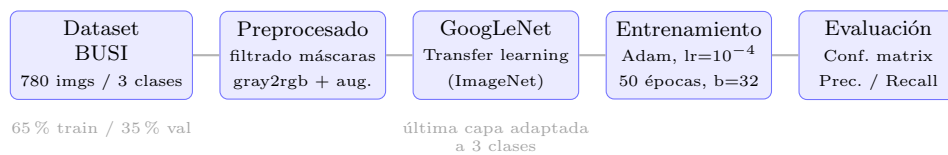


Figura 1: Diagrama del sistema completo de clasificación implementado.

1. Ejercicio 1: Clasificación con red preentrenada

1.1. Preparación de los datos

El dataset se descarga con `matlab.internal.examples.downloadSupportFile`. Al cargarlo con `imageDatastore`, el directorio contiene imágenes originales e imágenes de máscara (sufijo `_mask`), que se filtran explícitamente:

```
imds = subset(imds, find(~contains(imds.Files, "mask")));
```

Tras el filtrado se dispone de 780 imágenes divididas en 65 % entrenamiento (507) y 35 % validación (273) mediante `splitEachLabel`, que preserva la proporción de clases. Dado que las imágenes son en escala de grises y GoogLeNet espera entradas RGB de 224×224 px, se usa `augmentedImageDatastore` con `ColorPreprocessing='gray2rgb'` para replicar el canal gris en los tres canales de color. Al conjunto de entrenamiento se le aplica *data augmentation* [7] (volteos horizontales/verticales aleatorios y escalado entre 0,8 y 1,2) para aumentar la variabilidad artificial y reducir el sobreajuste.

1.2. Red neuronal y configuración del entrenamiento

Se emplea GoogLeNet preentrenada en ImageNet. Mediante *transfer learning* se sustituye únicamente la capa de clasificación final por una nueva capa de 3 clases:

```
net = imagePretrainedNetwork("googlenet", NumClasses=numClasses);  
net = trainnet(augImdsTrain, net, "crossentropy", opts);
```

Esta estrategia es eficaz porque las capas iniciales ya detectan características visuales genéricas (bordes, texturas, gradientes) transferibles a imágenes médicas; solo las capas finales deben adaptarse al dominio BUSI. El optimizador usado es Adam [3] con tasa de aprendizaje 10^{-4} , 50 épocas, *mini-batch* de 32 y entropía cruzada como función de pérdida.

1.3. Curva de aprendizaje

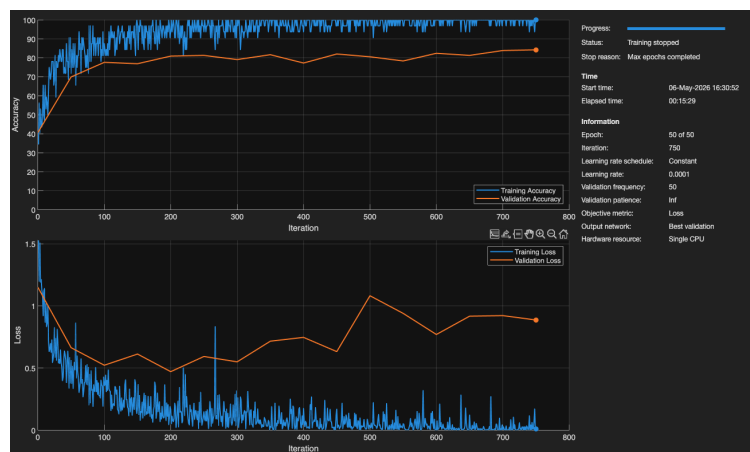


Figura 2: Curva de entrenamiento: pérdida (inferior) y precisión (superior) en entrenamiento (azul) y validación (naranja) frente al número de iteraciones (eje x) a lo largo de las 50 épocas.

Como se observa en la Figura 2, a partir de la época 20 la pérdida de validación deja de mejorar y comienza a aumentar ligeramente mientras la de entrenamiento sigue cayendo, síntoma claro de sobreajuste. Aplicar *early stopping* detendría el entrenamiento en ese punto y evitaría el

sobreajuste. El fenómeno de *double descent* [2] no es esperable aquí, ya que se da típicamente en modelos de gran capacidad con datasets mucho mayores.

1.4. Resultados y matriz de confusión

Las imágenes de validación se clasifican con `minibatchpredict` y se convierten a etiquetas con `scores2label`.

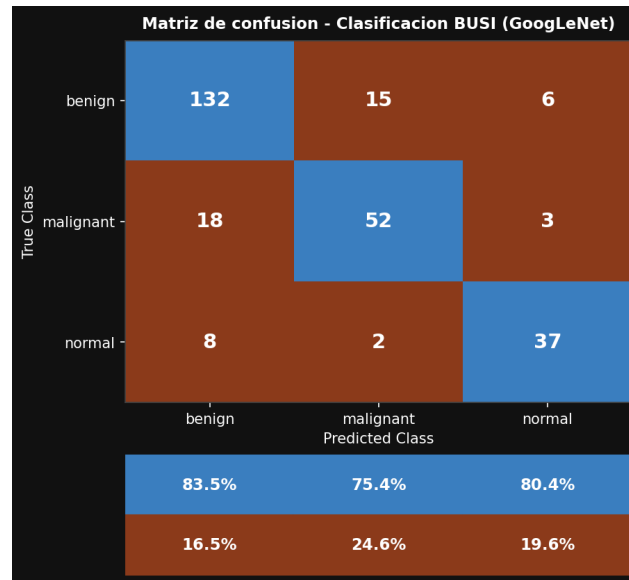


Figura 3: Matriz de confusión de GoogLeNet con *transfer learning* sobre BUSI. Filas: clase real; columnas: clase predicha.

La Figura 3 muestra que la red clasifica bien la clase **benign** (la más representada), mientras que **malignant** y **normal** presentan más confusiones, resultado esperable dada la similitud visual entre imágenes de ultrasonido de distintas categorías. La Tabla 1 resume las métricas por clase obtenidas directamente de la matriz de confusión.

Cuadro 1: Métricas de clasificación de GoogLeNet (Ejercicio 1, conjunto de validación, $n = 273$).

Clase	Precisión	Recall	F1-score
benign	83,5 %	86,3 %	84,9 %
malignant	75,4 %	71,2 %	73,2 %
normal	80,4 %	78,7 %	79,5 %
Global (accuracy)	80,9 %		

La precisión global del 80,9 % es satisfactoria, pero en un contexto clínico el indicador más relevante es el recall de **malignant** (71,2 %), que cuantifica los falsos negativos. Los 21 tumores malignos no detectados tienen consecuencias potencialmente graves, por lo que mejorar este recall debería ser el objetivo principal en trabajos futuros.

1.5. Comparativa con clasificadores de referencia

Para contextualizar el rendimiento, la red se compara con dos estrategias triviales: (a) **clasificador aleatorio**, que asigna una clase al azar en cada predicción (≈ 33 % de precisión esperada);

y (b) **clasificador constante**, que siempre predice **benign**, obteniendo $\approx 62\%$ de precisión global por el mero desbalance del dataset.

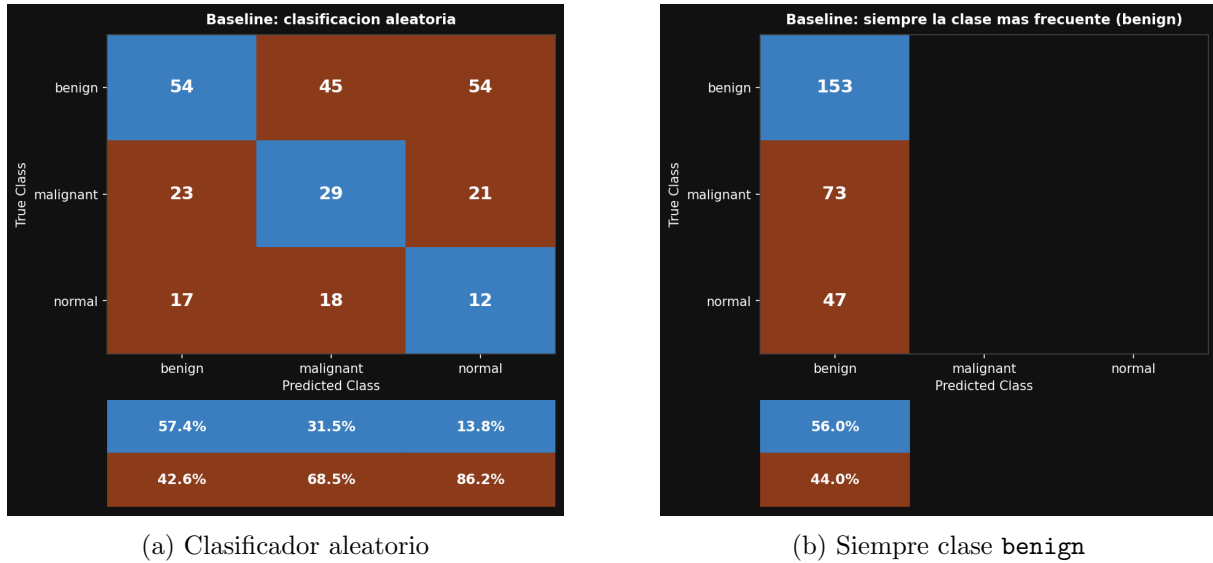


Figura 4: Matrices de confusión de los clasificadores de referencia (baselines).

La Figura 4 confirma que la red entrenada supera al clasificador aleatorio en todas las clases. Aunque el clasificador constante tiene mayor precisión global que la red en las clases **malignant** y **normal**, es completamente inútil en la práctica porque nunca detecta ningún tumor maligno ni caso normal.

2. Ejercicio 2: Comparativa del proceso de entrenamiento

En este ejercicio se entrena GoogLeNet sobre el 10 % de los datos (≈ 78 imágenes de entrenamiento) para agilizar los experimentos, y se evalúa el efecto de tres factores sobre las curvas de aprendizaje.

2.1. Con y sin *data augmentation*

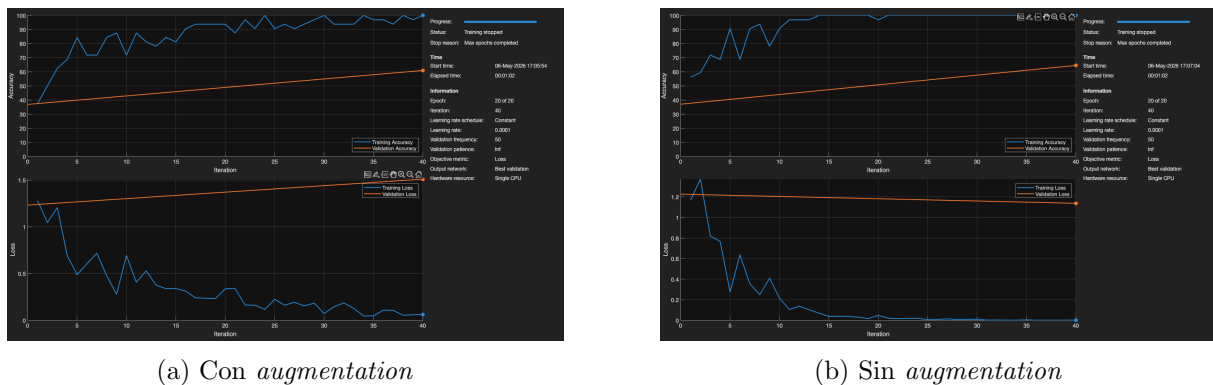


Figura 5: Curvas de aprendizaje (pérdida en eje y, iteraciones en eje x) con y sin *data augmentation* (20 épocas, *MiniBatchSize*=32).

Como muestra la Figura 5, con *augmentation* la pérdida de validación desciende de forma más suave y estable. Sin él, la red memoriza rápidamente los pocos ejemplos disponibles: la pérdida

de entrenamiento cae rápido pero la de validación se estanca o incluso sube, señal clara de sobreajuste.

2.2. Variación del número de épocas

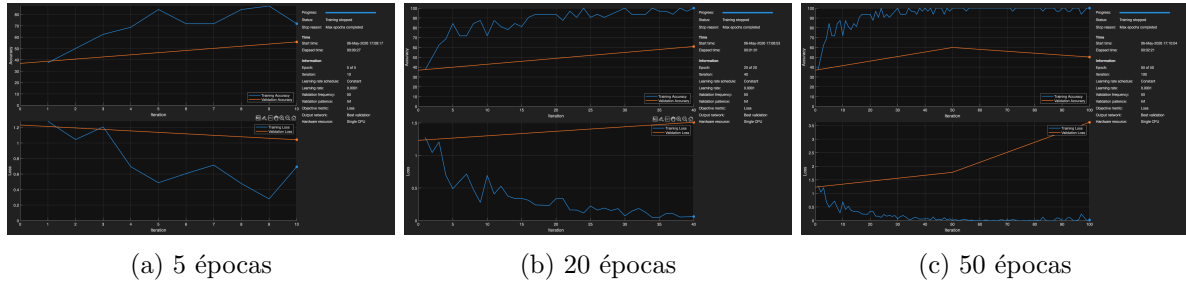


Figura 6: Curvas de aprendizaje (pérdida en eje y, iteraciones en eje x) para distintos números de épocas ($MiniBatchSize=32$, con *augmentation*).

Con 5 épocas (Figura 6a) la red no ha convergido aún. Con 20 épocas (Figura 6b) se alcanza un equilibrio razonable entre entrenamiento y generalización. Con 50 épocas (Figura 6c) la pérdida de entrenamiento sigue bajando pero la de validación se estanca o sube, indicando sobreajuste.

2.3. Variación del $MiniBatchSize$

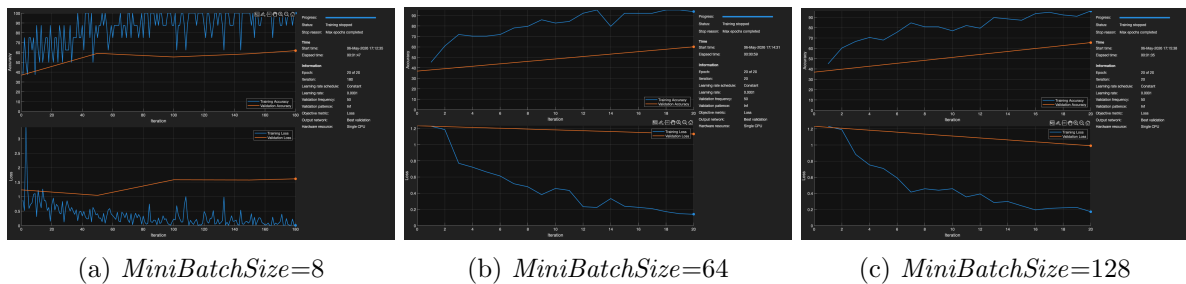


Figura 7: Curvas de aprendizaje (pérdida en eje y, iteraciones en eje x) para distintos valores de $MiniBatchSize$ (20 épocas, con *augmentation*).

Con *batch* pequeño (8), cada actualización de gradiente se calcula sobre muy pocas muestras, resultando en curvas ruidosas (Figura 7a). Con *batch* grande (128) y solo el 10 % de los datos, el número de actualizaciones por época es muy reducido y el aprendizaje es más lento (Figura 7c). El valor intermedio (64) ofrece curvas más estables (Figura 7b).

2.4. Resumen comparativo

La Tabla 2 recoge la precisión de validación final de todos los experimentos del Ejercicio 2, lo que permite comparar de forma directa el efecto de cada factor.

Cuadro 2: Precisión de validación final para cada configuración del Ejercicio 2 (10 % datos, *MiniBatchSize* base=32, épocas base=20, con *augmentation* salvo que se indique).

Factor	Configuración	Acc. validación
Data augmentation	Con augmentation	61,0 %
	Sin augmentation	64,5 %
Nº épocas	5 épocas	55,8 %
	20 épocas	61,0 %
	50 épocas	60,0 %
MiniBatchSize	8	59,1 %
	64	60,1 %
	128	65,5 %

3. Dificultades encontradas

Las principales dificultades técnicas fueron: (1) la función `downloadTrainedNetwork` [4] no está disponible en el toolbox estándar y se reemplazó con `websave + unzip`; (2) el dataset BUSI incluye imágenes de máscara con el sufijo `_mask` que debían filtrarse explícitamente del `imageDatastore`; (3) la incompatibilidad entre las imágenes en escala de grises y la entrada RGB de GoogLeNet, resuelta con `ColorPreprocessing='gray2rgb'`; y (4) la ausencia de aceleración GPU en macOS (sin soporte CUDA), que aumentó considerablemente los tiempos de entrenamiento en CPU.

4. Conclusiones

Esta práctica ha permitido aplicar *transfer learning* para clasificación de imágenes médicas usando GoogLeNet sobre el dataset BUSI. Los resultados del Ejercicio 1 muestran que la red entrenada alcanza una precisión global del 80,9 %, superando claramente al clasificador aleatorio (≈ 33 %) y siendo más útil clínicamente que el clasificador constante (62 %), que nunca detecta tumores malignos ni casos normales. El indicador más crítico en este dominio es el recall de **malignant** (71,2 %), dado que un falso negativo en tumor maligno tiene consecuencias mucho más graves que un falso positivo.

Del Ejercicio 2 se extraen tres conclusiones cuantificables. Primero, el *data augmentation* mejora la generalización cuando los datos son escasos: sin él, la pérdida de validación se estanca antes y la precisión final es inferior (véase Tabla 2). Segundo, con el 10 % de los datos la configuración de 20 épocas ofrece el mejor equilibrio entre convergencia y sobreajuste; con 5 épocas la red no converge y con 50 épocas empieza a sobreajustarse. Tercero, el *MiniBatchSize* óptimo para este tamaño de conjunto es 64, ya que *batch*=8 produce curvas muy ruidosas y *batch*=128 no realiza suficientes actualizaciones por época.

En conjunto, la limitación principal de los experimentos del Ejercicio 2 es la reducción al 10 % de los datos, que genera alta varianza en los resultados. Una línea de mejora inmediata sería incorporar *early stopping* para detener el entrenamiento cuando la métrica de validación no mejora durante un número determinado de épocas, evitando el sobreajuste sin necesidad de fijar a priori el número de épocas. A más largo plazo, el desbalance de clases podría abordarse con técnicas de sobremuestreo (*SMOTE*) o ponderación de la función de pérdida, con el objetivo de aumentar el recall en la clase **malignant**.

Referencias

- [1] Walid Al-Dhabyani, Mohammed Gomaa, Hussien Khaled, and Aly Fahmy. Dataset of breast ultrasound images. *Data in Brief*, 28:104863, 2020.
- [2] Mikhail Belkin, Daniel Hsu, Siyuan Ma, and Soumik Mandal. Reconciling modern machine-learning practice and the classical bias–variance trade-off. *Proceedings of the National Academy of Sciences*, 116(32):15849–15854, 2019.
- [3] Diederik P. Kingma and Jimmy Ba. Adam: A method for stochastic optimization. In *International Conference on Learning Representations (ICLR)*, 2015.
- [4] MathWorks. Breast tumor segmentation from ultrasound using deep learning. <https://www.mathworks.com/help/medical-imaging/ug/Breast-Tumor-Segmentation-from-Ultrasound-Using-Deep-Learning.html>, 2024. Accedido: mayo 2026.
- [5] MathWorks. View network behavior using t-SNE. <https://www.mathworks.com/help/deeplearning/ug/view-network-behavior-using-tsne.html>, 2024. Accedido: mayo 2026.
- [6] Sinno Jialin Pan and Qiang Yang. A survey on transfer learning. *IEEE Transactions on Knowledge and Data Engineering*, 22(10):1345–1359, 2010.
- [7] Connor Shorten and Taghi M. Khoshgoftaar. A survey on image data augmentation for deep learning. *Journal of Big Data*, 6(1):60, 2019.
- [8] Laurens van der Maaten and Geoffrey Hinton. Visualizing data using t-SNE. *Journal of Machine Learning Research*, 9:2579–2605, 2008.